COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE212 / Automating Server Management**	**SECTION:**	**CPE31S2**
DATE PERFORMED:	**Sept 10 2026**	**DATE SUBMITTED:**	**Sept 10 2026**
NAME:	**Kenzo Lim**	**INSTRUCTOR:**	**Engr. Jimlord Quejado**

ACTIVITY NO. 7.1 **Managing Files and Creating Roles in Ansible**

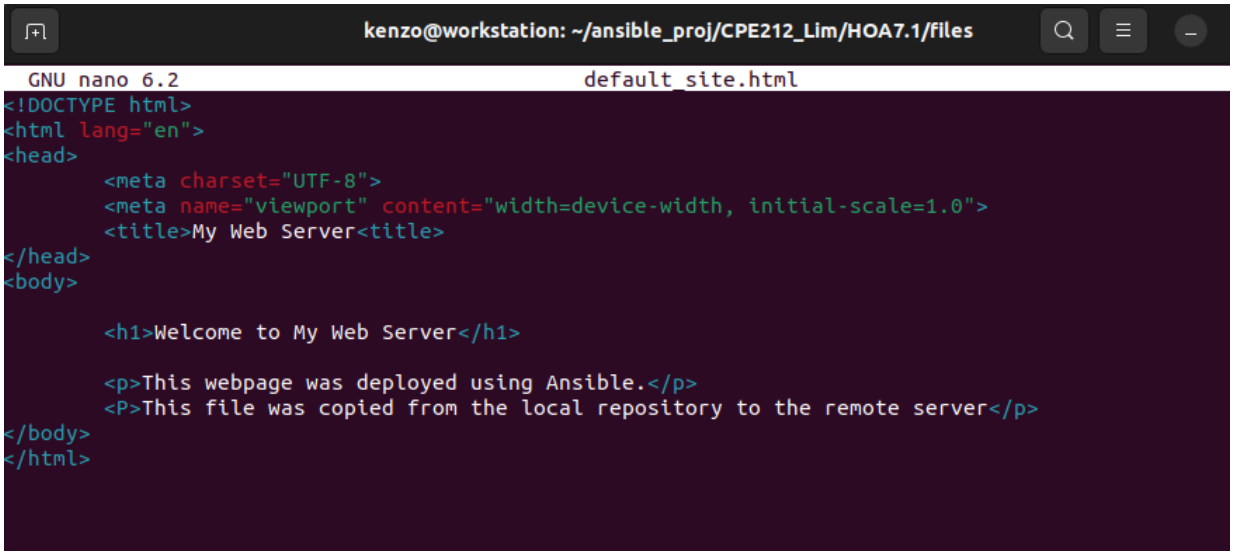
1. PROCEDURE OUTPUT

Task 1: Create a file and copy it to remote servers

1. Using the previous directory was created in your previous laboratory, create a directory, and name it “files.” Create a file inside that directory and name it “default_site.html.” Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Web Server</title>
</head>
<body>
<h1>Welcome to My Web Server</h1>
<p>This webpage was deployed using Ansible.</p>
<p>This file was copied from the local repository to the remote server.</p>
</body>
</html>
```



2. Edit the site.yml file and just below the web_servers play, create a new file to copy the default html file for site:

- name: copy default html file for site
tags: apache, apache2, httpd
copy:
src: files/default_site.html
dest: /var/www/html/index.html
owner: root
group: root
None
mode: '0644'

```
- name: copy default html file for site
  tags: apache, apache2, httpd
  copy:
    src: files/default_site.html
    dest: /var/www/html/index.html
    owner: root
    group: root
    mode: '0644'
```

3. Run the playbook site.yml. Describe the changes.

```
TASK [install mariadb package (CentOS)] ****************************************************************
skipping: [server1]
changed: [server2]

TASK [copy default html file for site] *****************************************************************
ok: [server1]
changed: [server2]
[WARNING]: Could not match supplied host pattern, ignoring: file_servers

PLAY [file_servers] ************************************************************************************
skipping: no hosts matched

PLAY RECAP *********************************************************************************************
server1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=4    rescued=0
 ignored=0
server2                    : ok=7    changed=2    unreachable=0    failed=0    skipped=1    rescued=0
 ignored=0
```

4. Go to the remote servers (web_servers) listed in your inventory. Use cat command to check if the index.html is the same as the local repository file (default_site.html). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output.

```
kenzo@workstation:~$ cat /var/www/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My Web Server<title>
</head>
<body>

        <h1>Welcome to My Web Server</h1>

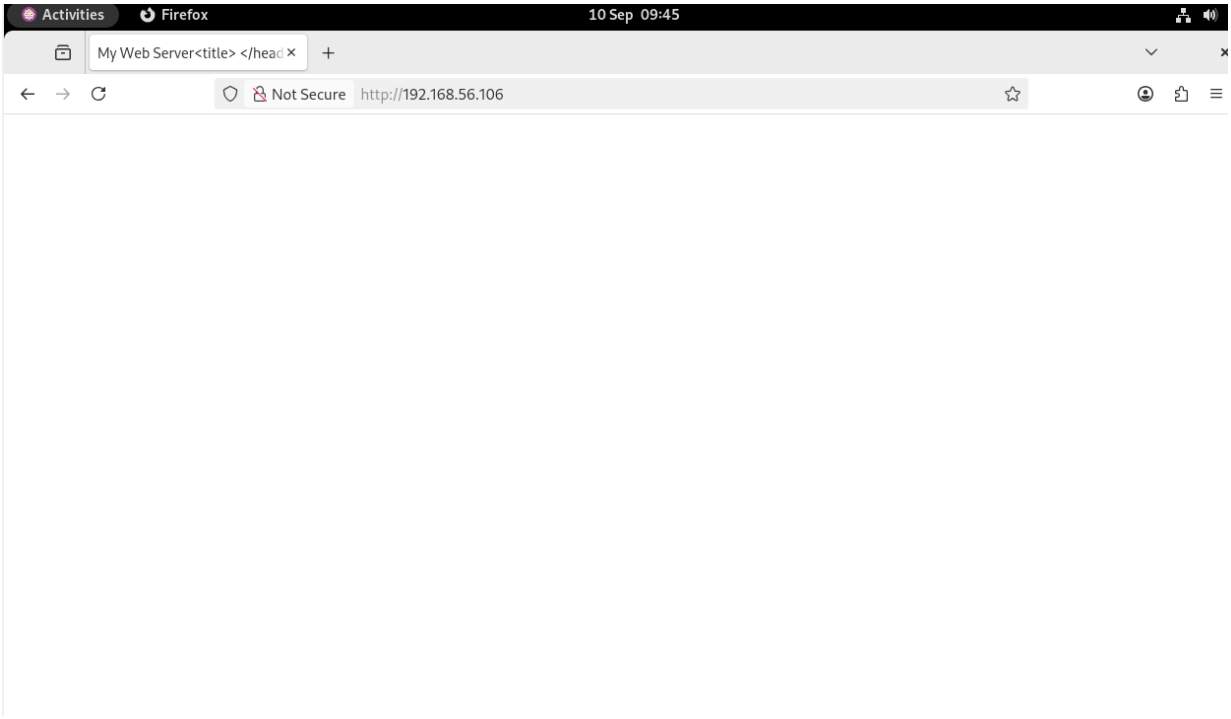
        <p>This webpage was deployed using Ansible.</p>
        <P>This file was copied from the local repository to the remote server</
p>
</body>
</html>
```

```
[kenzo@localhost ~]$ cat /var/www/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My Web Server<title>
</head>
<body>

        <h1>Welcome to My Web Server</h1>

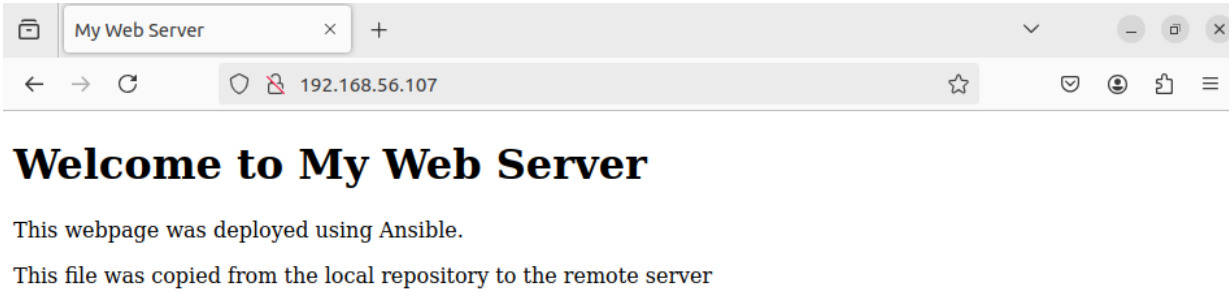
        <p>This webpage was deployed using Ansible.</p>
        <P>This file was copied from the local repository to the remote server</
p>
</body>
</html>
[kenzo@localhost ~]$
```

The default_file.html was copied to the destination of both Ubuntu and CentOS



This error was a result of not being able to end the <title></title> line in default_site.yaml

I was able to fix it later on and this was the result



5. Sync your local repository with GitHub and describe the changes.

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ git add *
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ git commit -m "HOA7.1 Procedure Task 1"
[main 2a42bcf] HOA7.1 Procedure Task 1
 3 files changed, 93 insertions(+)
 create mode 100644 HOA7.1/files/default_site.html
 create mode 100644 HOA7.1/inventory.yaml
 create mode 100644 HOA7.1/site.yaml
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 1.21 KiB | 618.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:qknlim/CPE212_Lim
   db793fb..2a42bcf  main -> main
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ ls
files  inventory.yaml  site.yaml
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$
```

Task 2: Download a file and extract it to a remote server

1. Edit the site.yml. Just before the web_servers play, create a new play:
- hosts: workstations
become: true
tasks:
- name: install unzip
package:
name: unzip
- name: install terraform
unarchive:
src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
dest: /usr/local/bin
remote_src: yes
mode: 0755
owner: root
group: root
note: In Ansible, a play is a section of a playbook that defines which remote machines Ansible should manage and what tasks it should perform on them.

kenzo@workstation: ~/ansible_proj/CPE212_Lim/HOA7.1

GNU nano 6.2 site.yaml *

```
- hosts: workstations
  become: true
  tasks:
   - name: install unzip
     package:
       name: unzip

   - name: install terraform
     unarchive:
       src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
       dest: /usr/local/bin
       remote_src: yes
       mode: '0755'
       owner: root
       group: root
```

2. Edit the inventory file and add a workstation group. Add any Ubuntu remote server. Make sure to remember the IP address.

kenzo@workstation: ~/ansible_proj/CPE212_Lim/HOA7.1

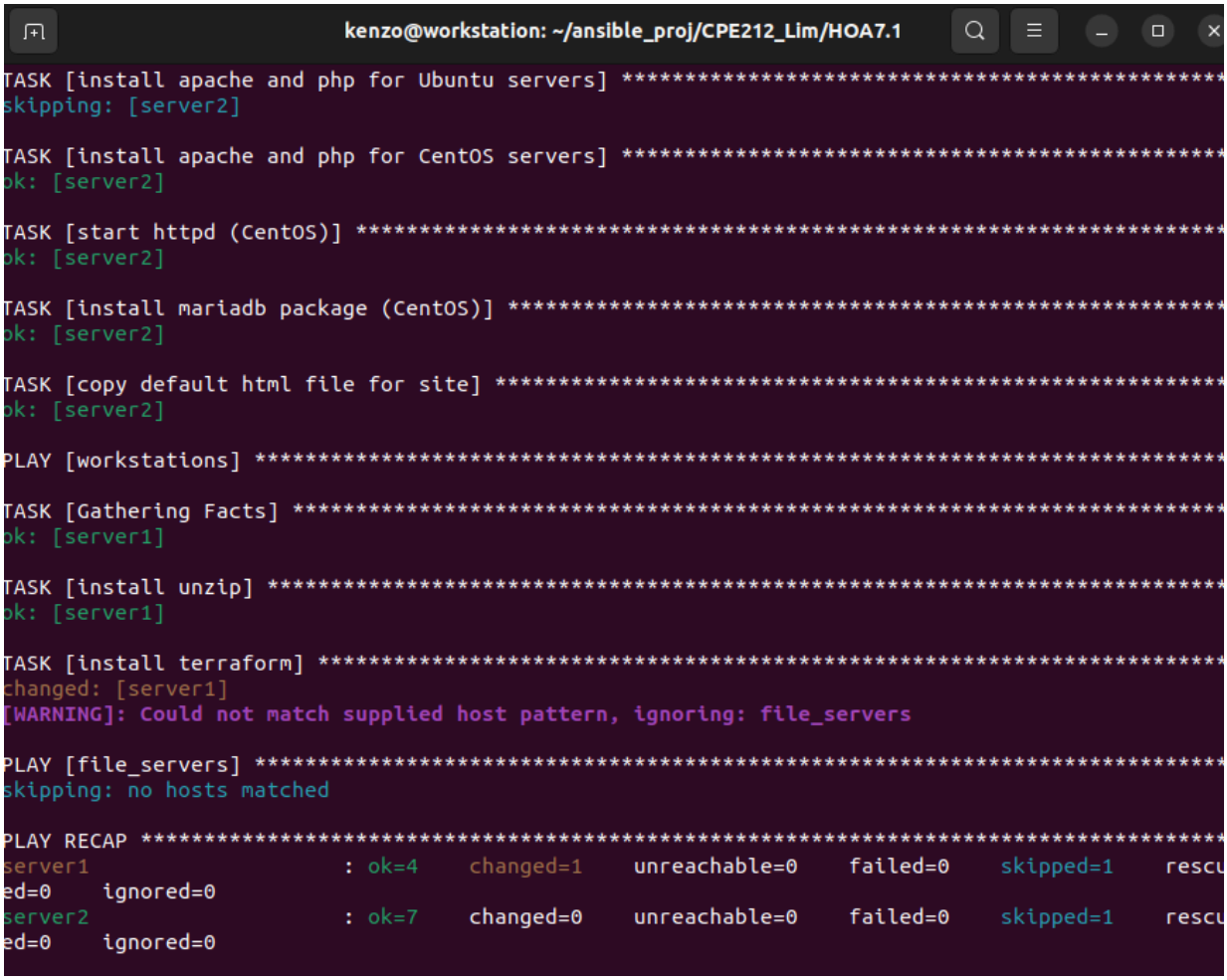
GNU nano 6.2 inventory.yaml

```
all:
  children:
    workstations:
      hosts:
        server1:
          ansible_host: kenzo@192.168.56.107
    web_servers:
      hosts:
        server2:
          ansible_host: kenzo@192.168.56.106
```

```
[web_servers]
192.168.56.106

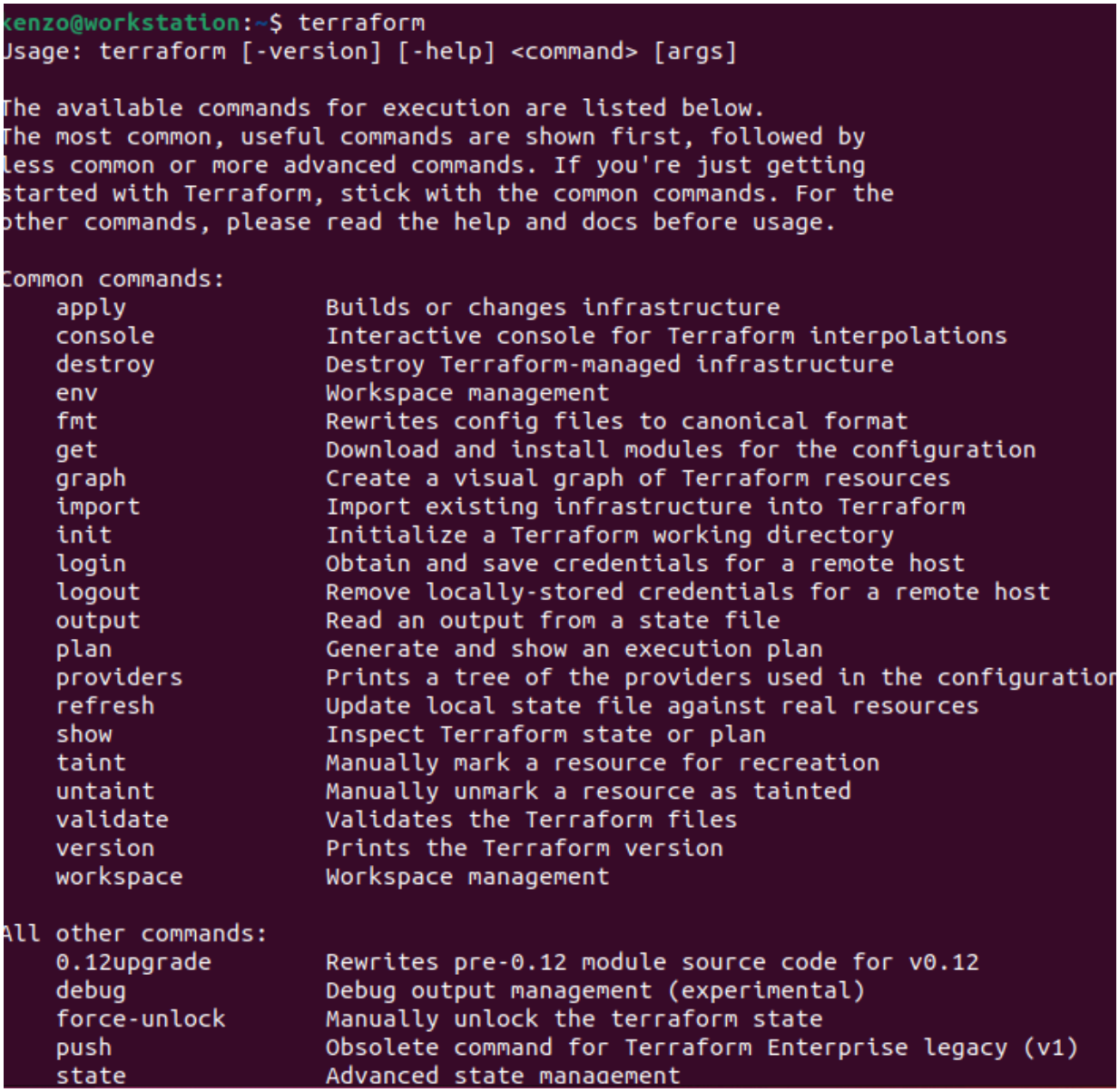

[workstations]
192.168.56.107
```

3. Run the playbook. Describe the output.



We can see that server 1 (workstation VMs) were changed, which meant that terraform was installed in the remote server 1

4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.



```
kenzo@workstation:~$ terraform
Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.

Common commands:
    apply              Builds or changes infrastructure
    console            Interactive console for Terraform interpolations
    destroy            Destroy Terraform-managed infrastructure
    env                Workspace management
    fmt                Rewrites config files to canonical format
    get                Download and install modules for the configuration
    graph              Create a visual graph of Terraform resources
    import             Import existing infrastructure into Terraform
    init               Initialize a Terraform working directory
    login              Obtain and save credentials for a remote host
    logout             Remove locally-stored credentials for a remote host
    output             Read an output from a state file
    plan               Generate and show an execution plan
    providers          Prints a tree of the providers used in the configuration
    refresh            Update local state file against real resources
    show               Inspect Terraform state or plan
    taint              Manually mark a resource for recreation
    untaint            Manually unmark a resource as tainted
    validate           Validates the Terraform files
    version            Prints the Terraform version
    workspace          Workspace management

All other commands:
    0.12upgrade        Rewrites pre-0.12 module source code for v0.12
    debug              Debug output management (experimental)
    force-unlock       Manually unlock the terraform state
    push               Obsolete command for Terraform Enterprise legacy (v1)
    state              Advanced state management
```

By typing terraform in the Command Line Interface inside the Workstation Server, it provided information regarding how to use terraform commands, which means that terraform was successfully installed in the Workstation Server.

Task 3: Create roles

1. Edit the site.yml. Configure roles as follows: (make sure to create a copy of the old site.yml file because you will be copying the specific plays for all groups)
Save the file and exit.

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ cp site.yaml site_copy.yaml
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ ls
files  inventory.yaml  site_copy.yaml  site.yaml
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ site_copy.yaml
```

2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ mkdir roles
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ cd roles
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ mkdir base
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ mkdir web_servers
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ mkdir file_servers
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ mkdir db_servers
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ mkdir workstations
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ ls
base  db_servers  file_servers  web_servers  workstations
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ cd base
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/base$ mkdir tasks
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/base$ cd ..
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ cd db_servers
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/db_servers$ mkdir tasks
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/db_servers$ cd ..
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ cd file_servers
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/file_servers$ mkdir tasks
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/file_servers$ cd ..
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ cd workstations
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/workstations$ mkdir tasks
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/workstations$ cd ..
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ cd web_servers
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/web_servers$ ls
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles/web_servers$ mkdir tasks
```

3. Go to tasks for all directory and create a file. Name it main.yml. In each of the tasks for all directories, copy and paste the code from the old site.yml file. Show all contents of main.yml files for all tasks.

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1/roles$ tree
.
├── base
│   └── tasks
│       └── main.yaml
├── db_servers
│   └── tasks
│       └── main.yaml
├── file_servers
│   └── tasks
│       └── main.yaml
├── web_servers
│   └── tasks
│       └── main.yaml
└── workstations
    └── tasks
        └── main.yaml
```

4. Run the site.yml playbook and describe the output.

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA7.1$ ansible-playbook -i inventory.yaml site_copy.yaml --
ask-become-pass
BECOME password:

PLAY [all] *********************************************************************************

TASK [Gathering Facts] *********************************************************************
[WARNING]: Platform linux on host server1 is using the discovered Python interpreter at
/usr/bin/python3.10, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [server1]
[WARNING]: Platform linux on host server2 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [server2]

TASK [update repository index (CentOS)] ****************************************************
skipping: [server1]
ok: [server2]

TASK [install updates (Ubuntu)] ************************************************************
skipping: [server2]
ok: [server1]

PLAY [all] *********************************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [server1]
ok: [server2]

TASK [base : install updates (CentOS)] *****************************************************
skipping: [server1]
ok: [server2]
```

```
TASK [base : install updates (CentOS)] ****************************************************
skipping: [server1]
ok: [server2]

TASK [base : install apache and php for Ubuntu servers] ***********************************
skipping: [server2]
ok: [server1]

TASK [base : install apache and php for CentOS servers] ***********************************
skipping: [server1]
ok: [server2]

TASK [base : start httpd (CentOS)] ********************************************************
skipping: [server1]
ok: [server2]

TASK [base : install mariadb package (CentOS)] ********************************************
skipping: [server1]
ok: [server2]

TASK [base : copy default html file for site] *********************************************
ok: [server1]
ok: [server2]

PLAY [workstations] ***********************************************************************

TASK [Gathering Facts] ********************************************************************
ok: [server1]

TASK [workstations : install updates (CentOS)] ********************************************
skipping: [server1]

TASK [workstations : install apache and php for Ubuntu servers] ***************************
ok: [server1]

TASK [workstations : install apache and php for CentOS servers] ***************************
skipping: [server1]
```

```
TASK [workstations : install mariadb package (CentOS)] *************************************
skipping: [server1]

TASK [workstations : copy default html file for site] **************************************
ok: [server1]

PLAY [web_servers] *************************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [server2]

TASK [web_servers : install updates (CentOS)] **********************************************
ok: [server2]

TASK [web_servers : install apache and php for Ubuntu servers] *****************************
skipping: [server2]

TASK [web_servers : install apache and php for CentOS servers] *****************************
ok: [server2]

TASK [web_servers : start httpd (CentOS)] **************************************************
ok: [server2]

TASK [web_servers : install mariadb package (CentOS)] **************************************
ok: [server2]

TASK [web_servers : copy default html file for site] ***************************************
ok: [server2]
[WARNING]: Could not match supplied host pattern, ignoring: db_servers

PLAY [db_servers] **************************************************************************
skipping: no hosts matched
[WARNING]: Could not match supplied host pattern, ignoring: file_servers

PLAY [file_servers] ************************************************************************
skipping: no hosts matched

PLAY RECAP *********************************************************************************
server1                    : ok=8    changed=0    unreachable=0    failed=0    skipped=9    rescued=0
 ignored=0
server2                    : ok=14   changed=0    unreachable=0    failed=0    skipped=3    rescued=0
 ignored=0
```

In this output, we can see that there are:
Server 1: ok=8 and skipped=9
Server 2: ok=14 and skipped 3
The reason it had many tasks was because I copied all the tasks into all the main.yaml of each role, which is an error on my side as I should have put only specific tasks on each role. Understanding and encountering this mistake was able to make me realize that a role executes only certain tasks given inside the main.yaml file for each respective tasks directory.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of creating roles?
Roles help us organize our Ansible playbooks into smaller pieces. Instead of putting everything in one long playbook, we are able to break tasks into folders based on what they do (like web_servers, db_servers, workstations). This makes it easier to find and fix things, reuse the same role in different projects, and keep each server group doing only the tasks meant for it.

2. What is the importance of managing files?

Managing files properly makes sure the right files go to the right servers, with the right permissions and ownership. This keeps our setup consistent across all the servers, so every machine ends up configured the same way without having to do it by hand.

Conclusions/Learnings:

In this activity, I learned why roles and file management matter in Ansible. Roles help keep playbooks neat and organized by splitting tasks into separate folders. This makes it easier to reuse tasks and manage different servers. Managing files well helps make sure every server gets set up the same way without doing it manually.

One important lesson I learned was that each role should only have the tasks it actually needs inside its main.yml file. At first, I copied all the tasks into every role by mistake. This caused many tasks to be skipped or repeated when I ran the playbook. This showed me that roles work best when each one only has the tasks meant for it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.